Day 1: Foundations & Prompt Engineering

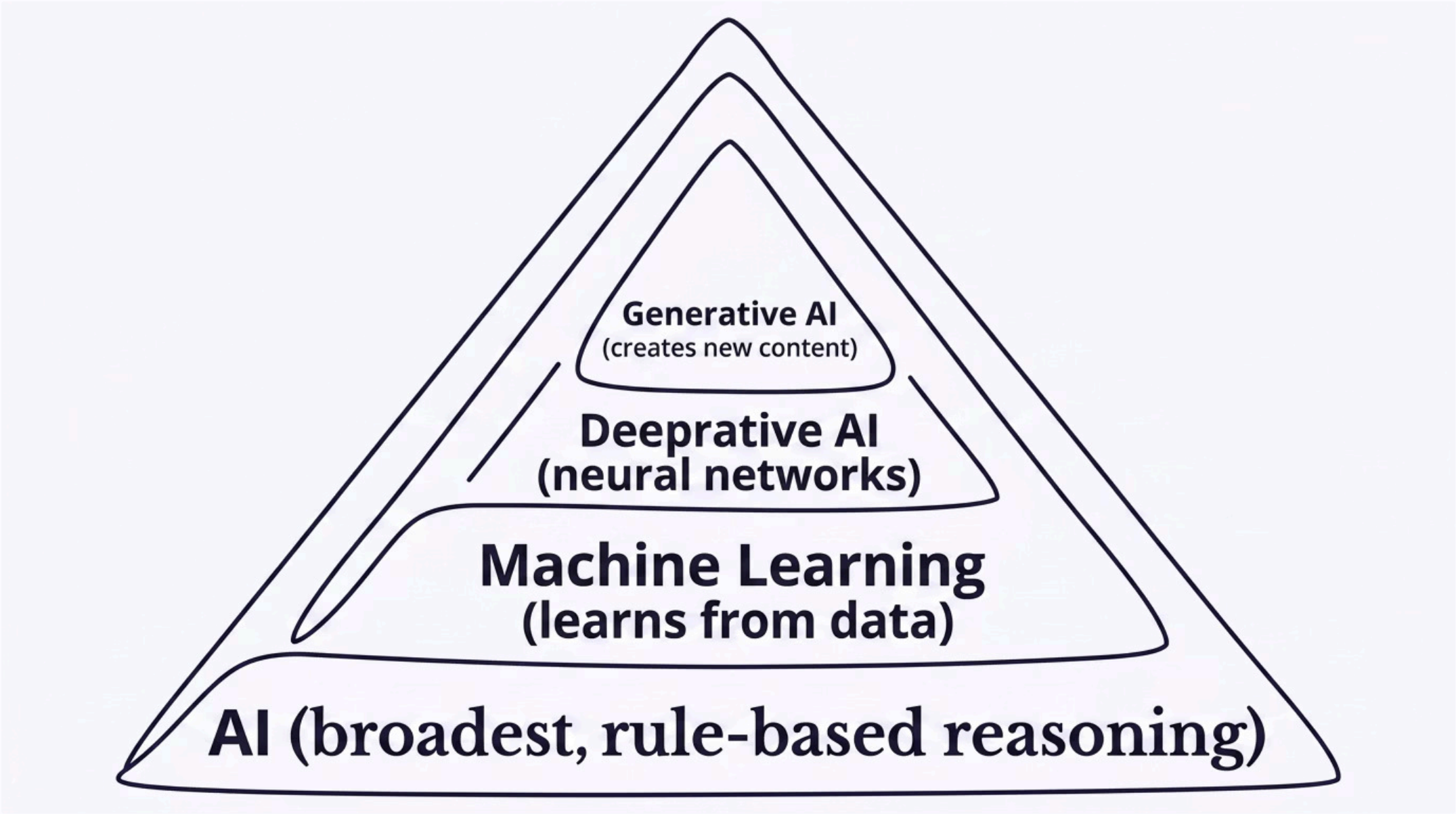
A hands-on introduction to Generative AI — from the fundamentals of how large language models work to the art and science of crafting effective prompts.

1. GenAI Fundamentals
2. How LLMs Work
3. Limitations
4. Prompt Engineering
5. Hands-on Lab

CHAPTER 1

The AI Landscape & Context

Artificial Intelligence is not a monolith — it is a layered hierarchy of technologies, each building on the last. Understanding this structure helps clarify what Generative AI actually is, and what makes it distinct from earlier approaches.



Predictive AI

Analyses existing data to **classify** or **forecast** outcomes. Examples: spam filters, fraud detection, recommendation engines.

Generative AI

Goes beyond prediction to **create new content** — text, images, code, audio — that did not previously exist. This is the paradigm shift.

CHAPTER 1

What is Generative AI?

Generative AI refers to models that can produce original content across multiple modalities — given a prompt, they synthesise something new rather than retrieving a stored answer.

Chatbots & Conversation

Engage in natural, context-aware dialogue. Power customer support, internal assistants, and intelligent search interfaces.

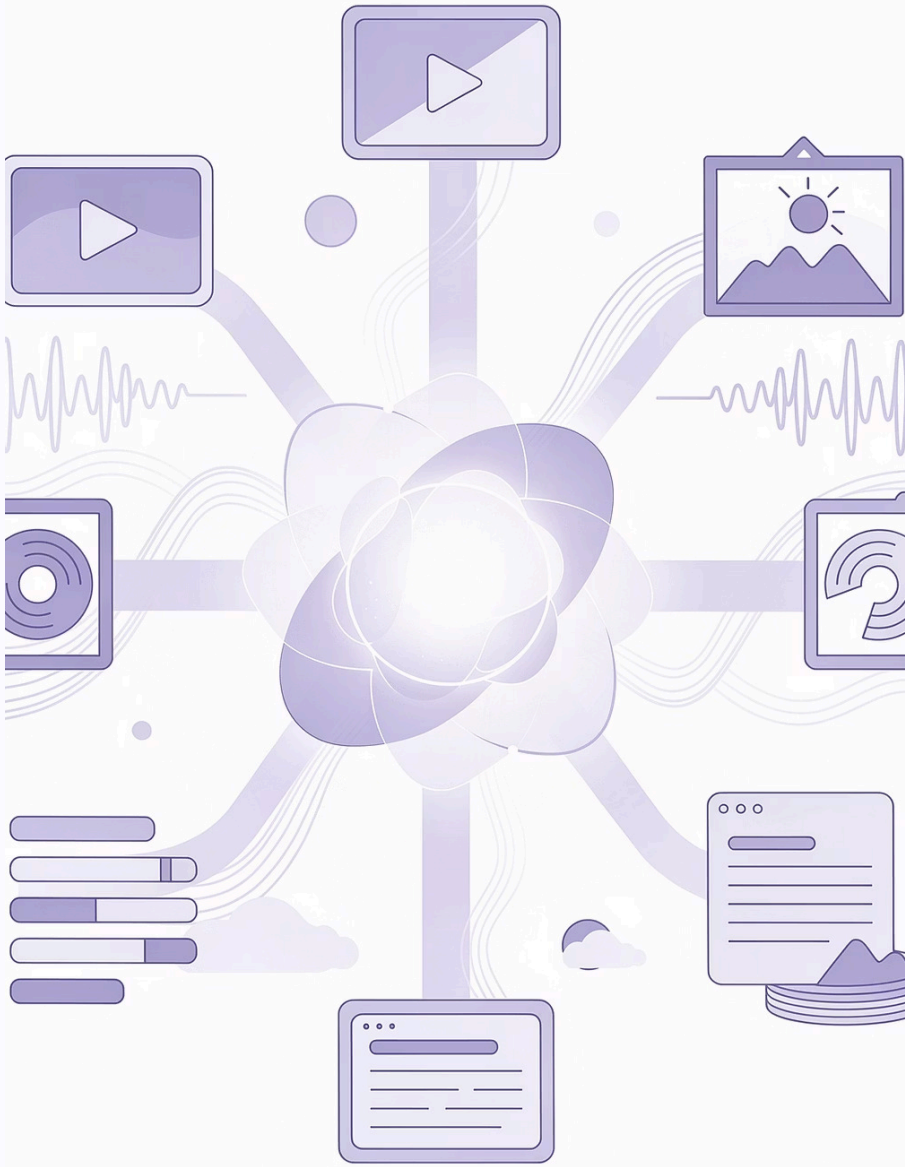
Summarisation

Condense lengthy documents, meeting transcripts, or research papers into clear, actionable summaries in seconds.

Code Generation

Write, explain, refactor, and debug code across dozens of programming languages — accelerating developer productivity.

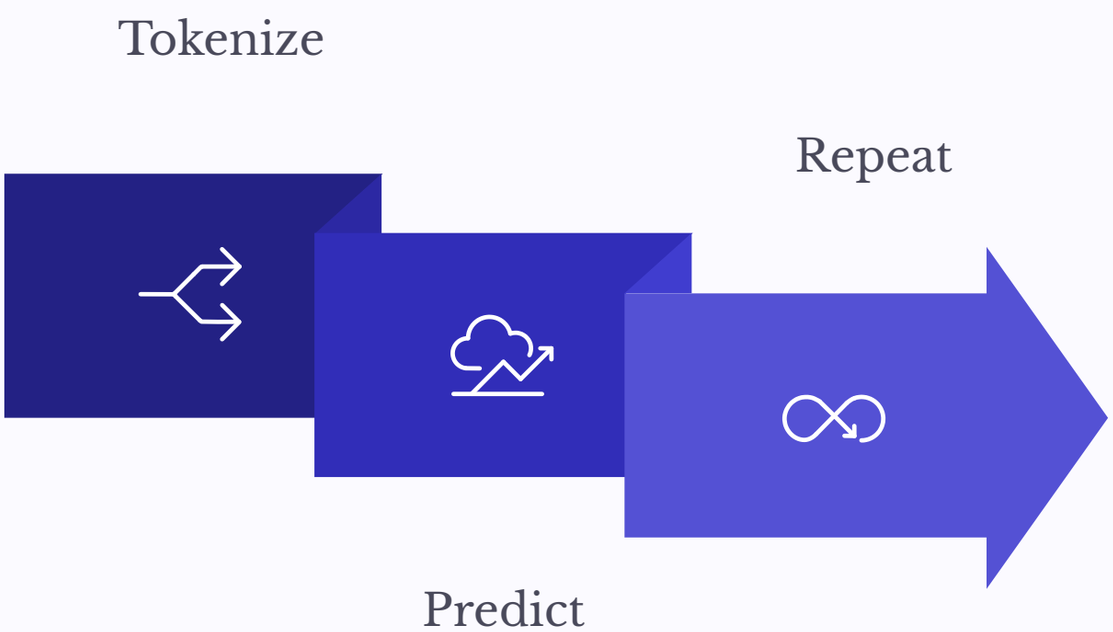
Key Anchor: LLMs are **probabilistic generators**, not knowledge engines. They do not look things up — they predict what comes next.



CHAPTER 2

How LLMs Work: The Core Mental Model

At their heart, large language models perform one deceptively simple task repeatedly: given everything seen so far, predict the most likely next token. Repeated thousands of times, this produces fluent, coherent text.



This architecture — trained on vast corpora of human-written text — gives LLMs their remarkable versatility. But it also explains their core limitation: they are **next-word prediction at scale**, not reasoning engines with ground truth access.

CHAPTER 2

Tokens, Probability & Decoding

Understanding how LLMs sample their output is essential for controlling the quality and style of responses. The model does not simply pick the "best" word — it samples from a probability distribution, shaped by decoding parameters you can tune.

Key Concepts

- **Tokens ≠ Words** — a token is roughly 3–4 characters. "unbelievable" = 3 tokens.
- **Probability distribution** — the model assigns a likelihood to every possible next token in the vocabulary.
- **Context window** — the total tokens (input + output) the model can hold in memory at once.

Decoding Parameters

- **Temperature** — controls creativity. Low = focused and deterministic. High = diverse and surprising.
- **Top-k** — restricts sampling to the k most likely next tokens.
- **Top-p (nucleus)** — samples from the smallest set of tokens whose cumulative probability exceeds p.

💡 **Insight:** Model output is **controlled randomness** — deterministic architecture producing non-deterministic results. The same prompt can yield different outputs on every run.

CHAPTER 3

Capabilities vs Limitations

LLMs are powerful generalists — but knowing where they excel and where they fail is critical to building reliable AI-powered products. Set the right expectations from the start.

✅ Strong Capabilities

- **Summarisation** — distilling large volumes of text into concise outputs
- **Transformation** — rewriting, translating, reformatting content
- **Code assistance** — generating boilerplate, explaining logic, fixing bugs
- **Brainstorming** — generating options, outlines, and creative drafts

⚠️ Known Struggles

- **Factual accuracy** — models can confidently state incorrect information
- **Long context** — performance degrades over very long documents
- **Real-time knowledge** — training data has a cut-off; models are not connected to live information
- **Precise arithmetic** — numerical reasoning is error-prone without tools

CHAPTER 3

Hallucination & the Need for RAG

Hallucination is one of the most discussed — and most misunderstood — failure modes of LLMs. It refers to the model generating plausible-sounding but factually incorrect content, presented with full confidence.

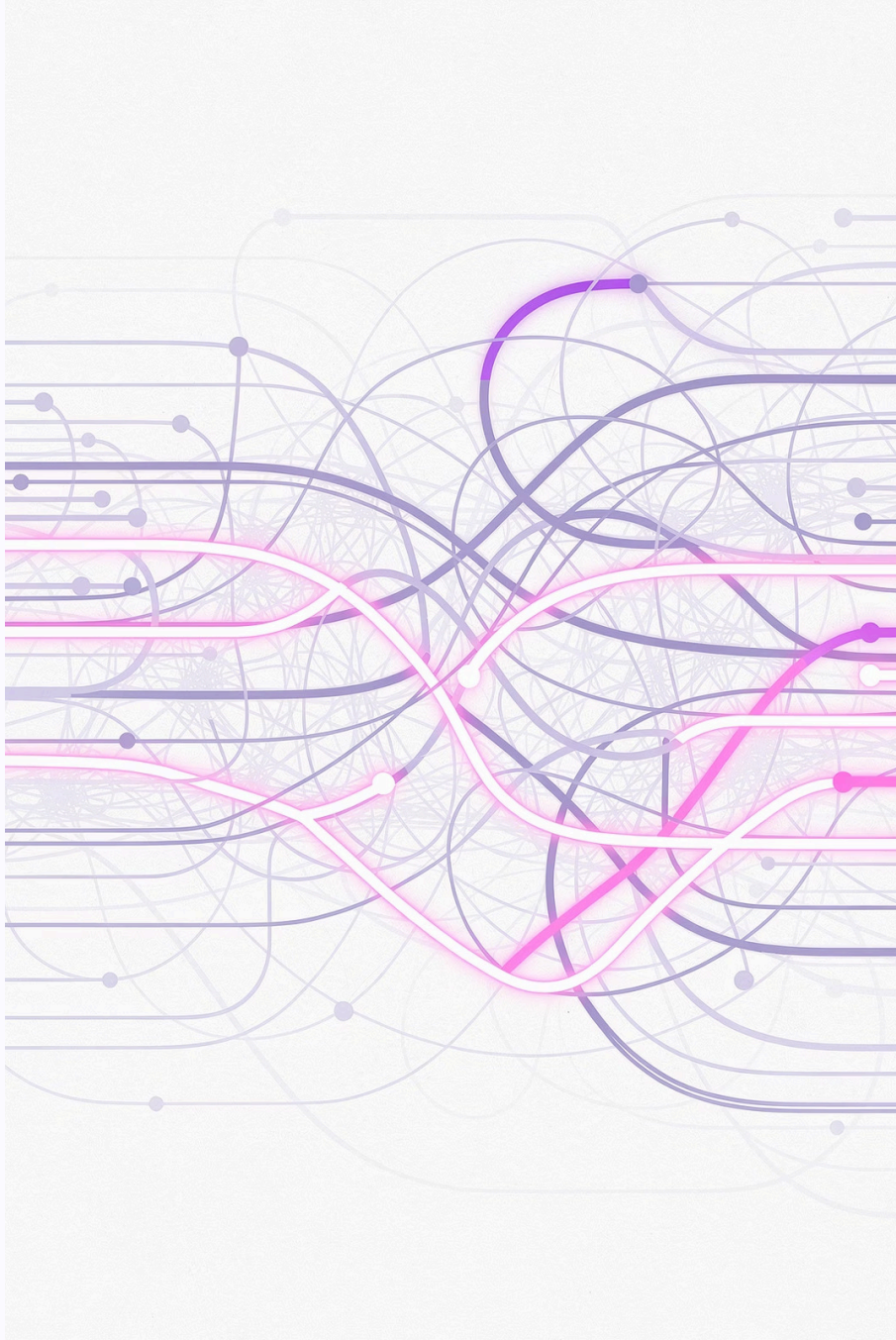
Why Hallucination Happens

- **No grounding** — the model has no access to verified facts; it predicts based on patterns alone
- **Ambiguity** — vague prompts cause the model to "fill in the gaps" with invented detail
- **Training artefacts** — rare or conflicting training data leads to unreliable outputs on niche topics

The RAG Solution

Retrieval-Augmented Generation grounds the model in real, verified documents at query time.

Query Retrieve Generate



CHAPTER 4

Prompt Engineering Fundamentals

Prompts are your primary control mechanism for LLM behaviour. A well-structured prompt is the difference between a vague, generic response and a precise, actionable output. **Treat prompts like code** — version them, test them, iterate on them.

1

Instruction

Tell the model **exactly what to do**. Be specific about the task, verb, and scope. Avoid ambiguity.

2

Context

Provide **background information** the model needs to perform the task correctly — persona, domain, constraints.

3

Input

Supply the **actual content** to act upon — the document, question, data, or text to be processed.

4

Output Format

Specify **how the response should be structured** — bullet points, JSON, a table, a paragraph, a specific length.

CHAPTER 4

Prompt Techniques

Beyond basic structure, a toolkit of prompting strategies dramatically improves output quality. Each technique serves a different purpose — knowing when to apply which is the mark of an effective prompt engineer.

Zero-Shot Prompting

Give the task directly with no examples. Works well for simple, well-defined tasks where the model has strong prior training.

Few-Shot Prompting

Provide 2–5 input/output examples before your actual request. Dramatically improves consistency for complex or stylised outputs.

Role Prompting

Assign the model a persona: *"You are a senior security engineer..."* Shapes tone, depth, and domain focus of the response.

Structured Output (JSON)

Instruct the model to respond in a defined schema. Essential for downstream integrations, APIs, and automated pipelines.

❌ Weak Prompt

"Summarise this article." — No context, no format, no constraints. Output will be unpredictable.

✅ Strong Prompt

"You are a technical writer. Summarise the following article in 3 bullet points for a non-technical audience. Focus on business impact."

HANDS-ON LAB

Lab & Wrap-up: Prompt Playground

Time to apply everything from today. In this lab you will move from observer to practitioner — building, testing, and refining prompts in a live environment.

🤖 Build a Chatbot

Design a system prompt that gives your LLM a consistent persona, tone, and set of constraints. Test it under varied user inputs.

🧪 Experiment with Prompts

Compare zero-shot vs few-shot, try role prompting, and adjust temperature settings. Document what changes and why.

📋 Structured Outputs

Generate JSON, tables, and bullet-structured data using explicit output format instructions. Validate the schema.

Day 1 Key Takeaway: LLMs are probabilistic generators controlled by context. Your prompt is your interface — engineer it with the same rigour you would apply to any production code.